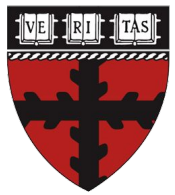
Lecture 3: Containers II

AC215/E115

Pavlos Protopapas
SEAS/Harvard



Outline

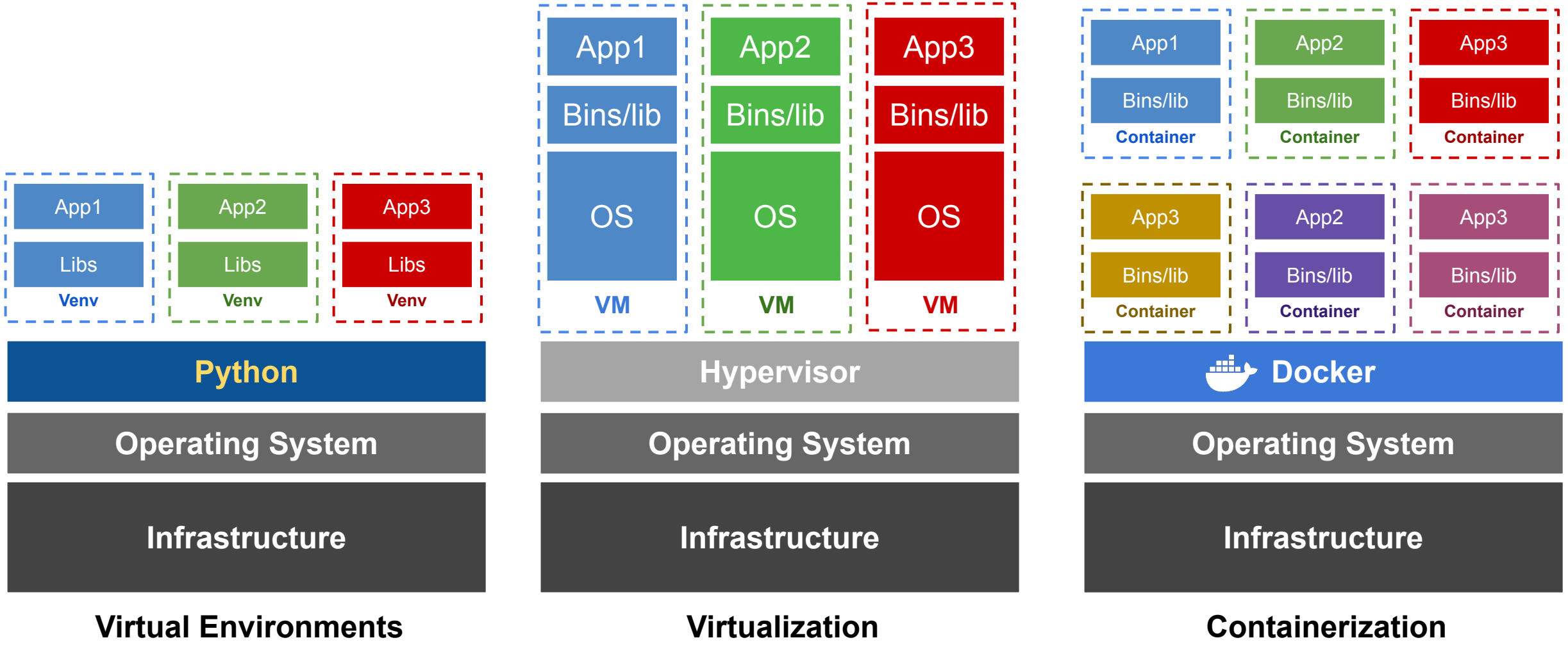
1. Recap: Review of Previous Material
2. Containers in Architecture: Microservices vs. Monolithic
3. Tutorial
4. Build Optimization
5. Docker Compose
6. Networking
7. Team Workflows

Outline

1. **Recap: Review of Previous Material**
2. Containers in Architecture: Microservices vs. Monolithic
3. Tutorial
4. Build Optimization
5. Docker Compose
6. Networking
7. Team Workflows



Recap: Environments vs Virtualization vs Containerization



Outline

1. Recap: Review of Previous Material
2. **Containers in Architecture: Microservices vs. Monolithic**
3. Tutorial
4. Build Optimization
5. Docker Compose
6. Networking
7. Team Workflows



Why use Containers?

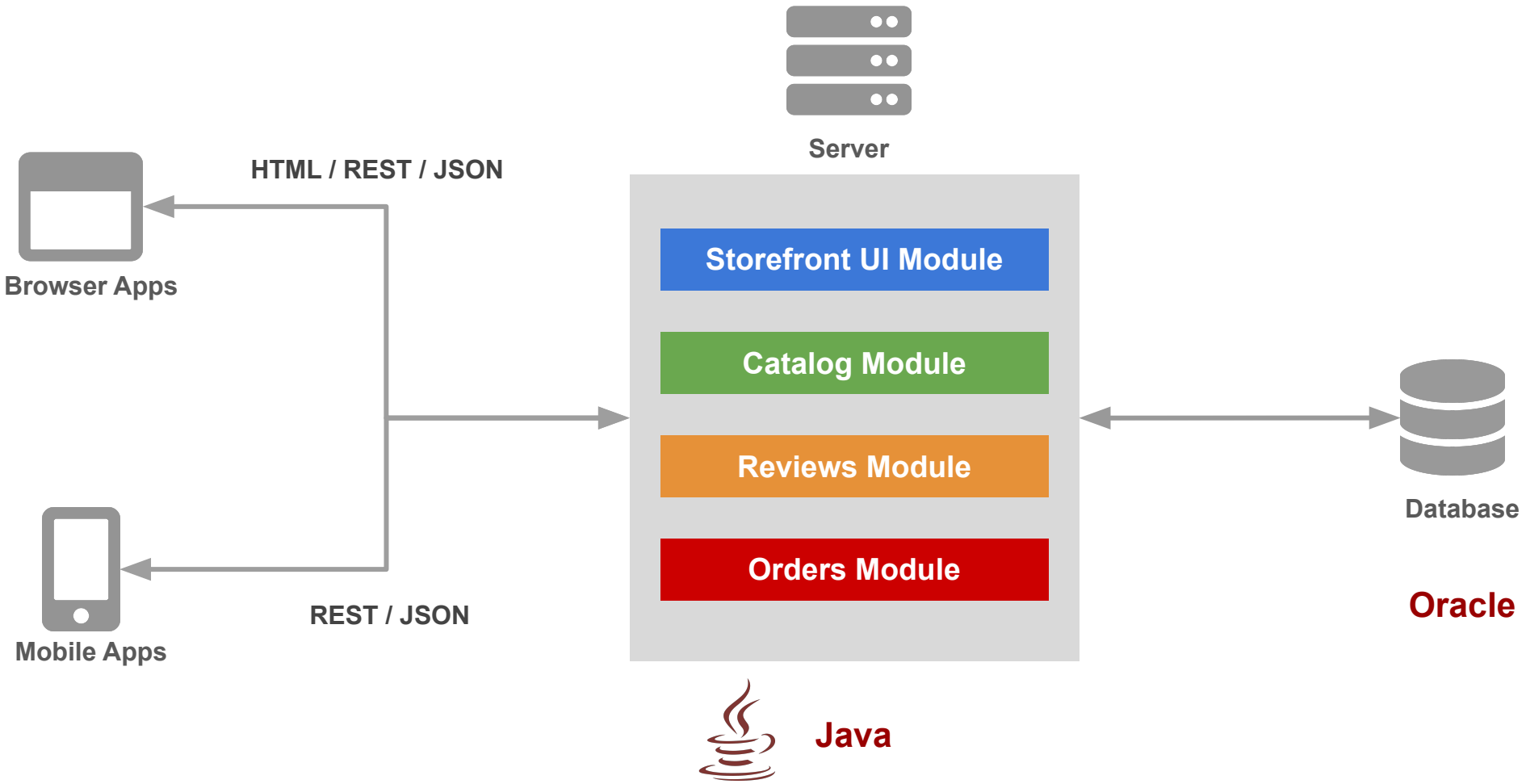
Conceptual Scenario

- Picture building an application, such as an online cheese store.

Traditional Approach

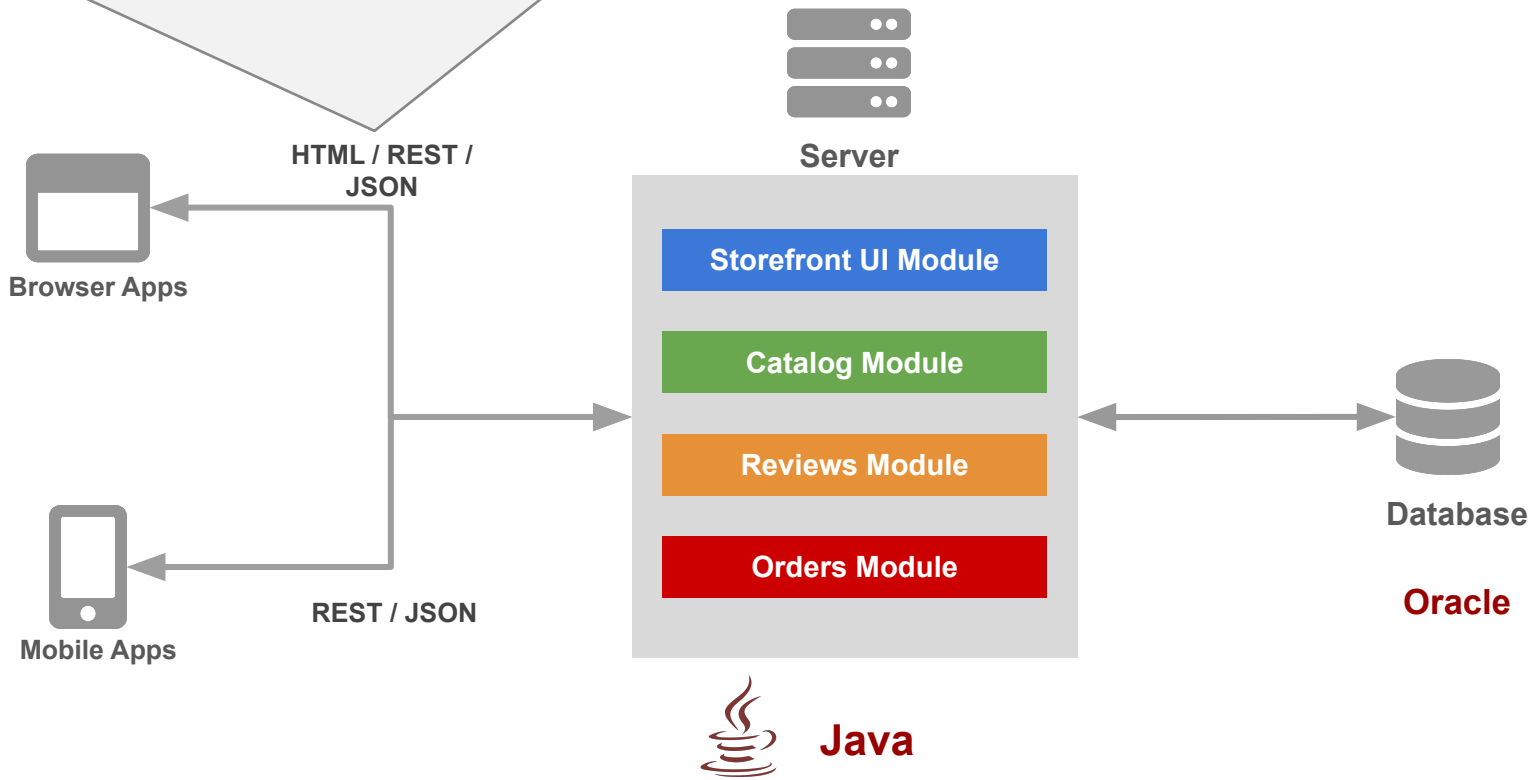
- Traditionality you would build this using a Monolithic Architecture.

Monolithic Architecture



Monolithic Architecture

HTML is HTML.
JSON represents data exchanged between clients and APIs.
REST is an architectural style for APIs.
HTTP is the communication protocol used here.



Monolithic Architecture - **Advantages**

Simplicity in Development:

Most tools and IDEs natively support monolithic applications.

Ease of Deployment:

All components bundled into a single, unified package.

Scalability:

Easier to scale by replicating the entire application as a whole (horizontal scaling).

Monolithic Architecture - **Disadvantages**

Maintenance Challenges:

Complexity increases over time, making it harder to implement changes or find issues.

System Vulnerability:

A failure in a single component can lead to the collapse of the entire system.

Patching Difficulties:

Patching or updating specific modules can be cumbersome due to tightly-coupled components.

Monolithic Architecture - **Disadvantages**

Technology Lock-in:

Adopting new technologies or updating existing ones can be problematic due to interdependencies.

Slow Startup:

Increased startup time as all components must be initialized simultaneously.

Onboarding Challenges:

New users need to familiarize themselves with the entire codebase.

Applications have changed dramatically

A decade ago

Apps were monolithic
Built on a single stack (e.g. .Net or Java)
Long lived
Deployed to a single server

Today

Apps are constantly being developed
Build from loosely coupled components
Newer version are deployed often
Deployed to a multitude of servers

Applications have changed dramatically

A decade ago

Apps were monolithic
Built on a single stack (e.g. .Net or Java)
Long lived
Deployed to a single server



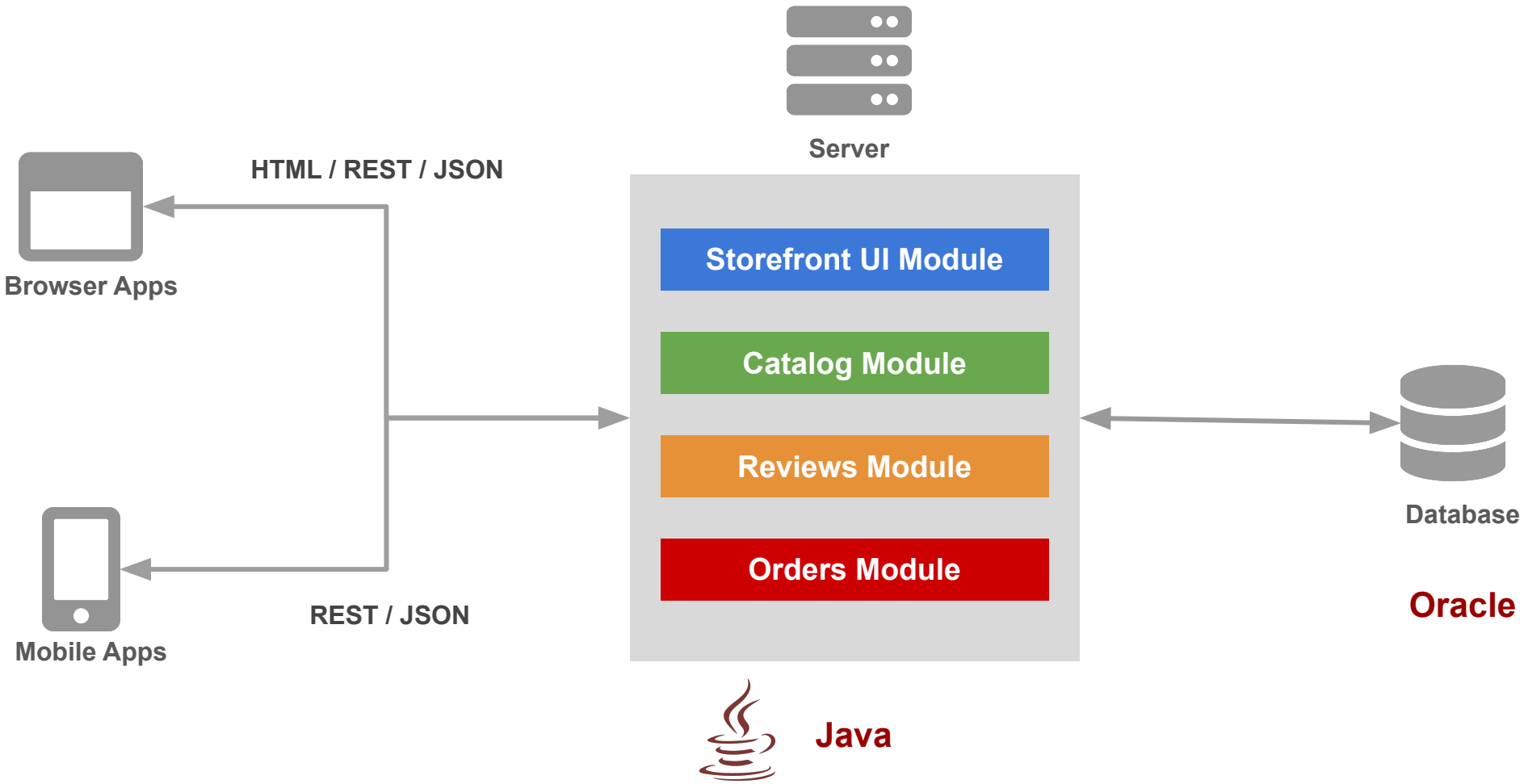
Today

Apps are constantly being developed
Build from loosely coupled components
Newer version are deployed often
Deployed to a multitude of servers

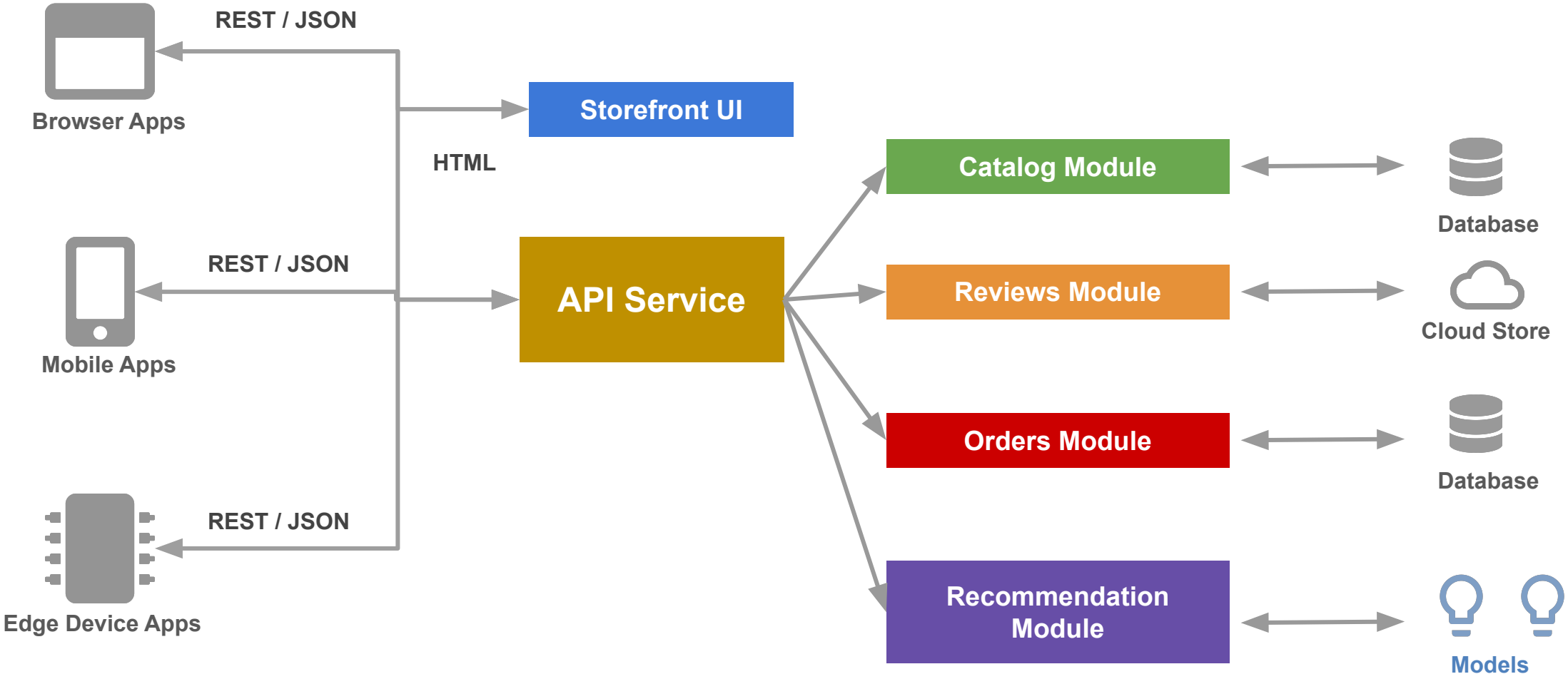
Data Science

Apps are being integrated with various data types/sources and models

Monolithic Architecture



Today: Microservice Architecture



Microservice Architecture - **Advantages**

Simplified Maintenance:

Modular design makes it easier to manage, update, and debug individual services.

Fault Isolation:

Independent components reduce the risk that a failure in one service brings down the entire application.

Streamlined Patching:

Easier to patch or update specific services without affecting the entire system.

Technological Flexibility:

Adapting to or adopting new technologies becomes seamless due to service independence.

Quick Startup:

Reduced startup time as all components can be initialized in parallel.

Microservice Architecture - **Disadvantages**

Development and Deployment Complexity:

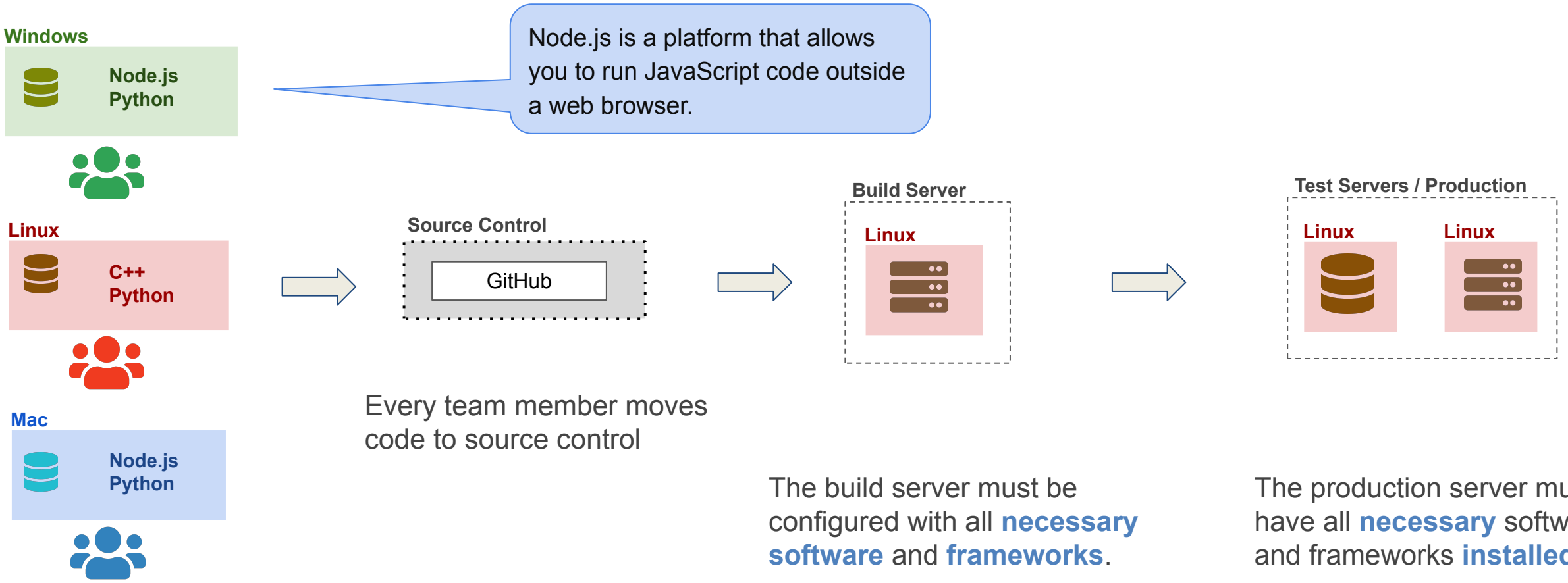
Using multiple technologies across components can complicate both development and deployment, as managing diverse dependencies requires a more intricate setup.

Scaling Concerns: Scaling the entire application can be intricate due to disparate components.



Docker + Kubernetes

Software Development Workflow (no Docker)



Every team member moves code to source control

OS Specific **installation** in every developer machine

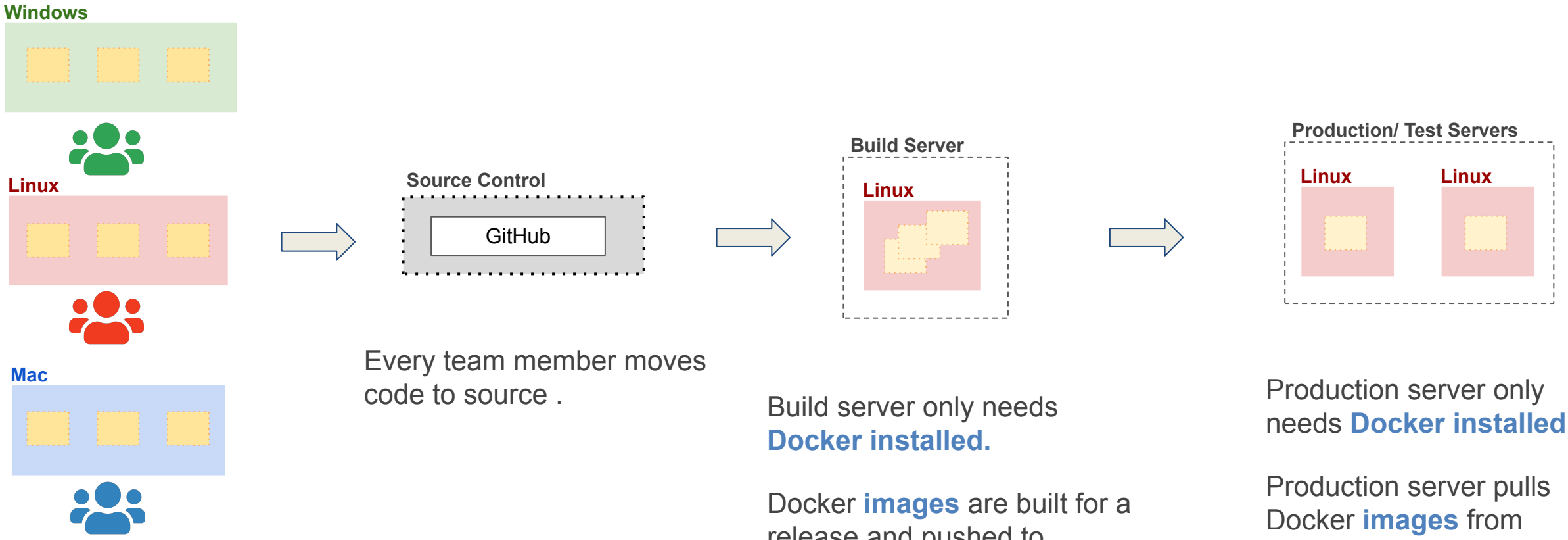
The build server must be configured with all **necessary software** and **frameworks**.

The production build process involves pulling the code from the source control system.

The production server must have all **necessary** software and frameworks **installed**.

It will also run on a different OS version compared to the development machines.

Software Development Workflow (with Docker)



Development machines only needs **Docker installed.**

Containers need to be setup only once.

Every team member moves code to source .

Build server only needs **Docker installed.**

Docker **images** are built for a release and pushed to **container registry.**

Production server only needs **Docker installed.**

Production server pulls Docker **images** from **container registry** and runs them.

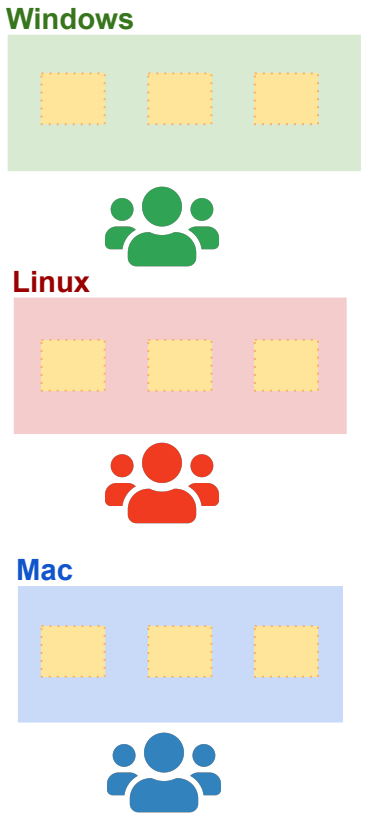
Software Development Workflow (with Docker)

Who creates the Dockerfile, and where is it stored? Do we use pre-built images or does each developer build them? Who is in charge of managing this? Also, what's the process for handling the pyproject.toml and uv.lock?

This seems like a lot.



Software Development Workflow (with Docker)



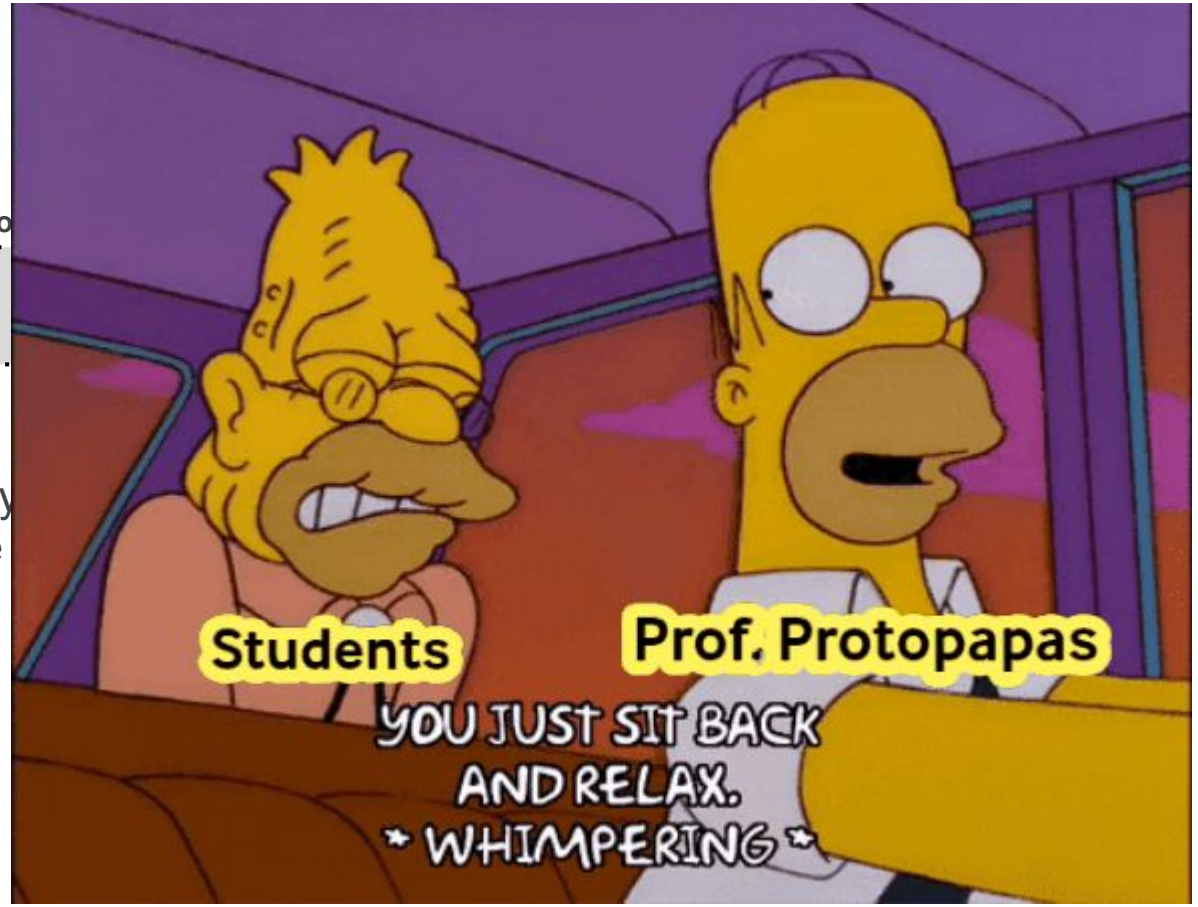
Development machines only needs **Docker installed.**

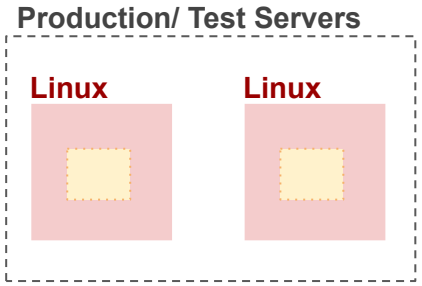
Containers need to be setup only once.



So

Every
code





Production server only needs **Docker installed.**

Production server pulls Docker **images** from **container registry** and runs them.

Outline

1. Recap: Review of Previous Material
2. Containers in Architecture: Microservices vs. Monolithic
3. **Tutorial**
4. Build Optimization
5. Docker Compose
6. Networking
7. Team Workflows

GCP Authentication Methods

Before we start let us review how do we authenticate to different services/accounts and APIs

OAuth 2.0:

For user-driven authentication and access.

Service Account (part of IAM in GCP):

For server-to-server interactions requiring automation and high control without user intervention.

API Key:

For lightweight, less secure access to APIs, use cautiously.

Default Service Accounts (part of IAM in GCP):

For Compute Engine, Kubernetes Engine, and App Engine with predefined permissions.

Workload Identity Federation:

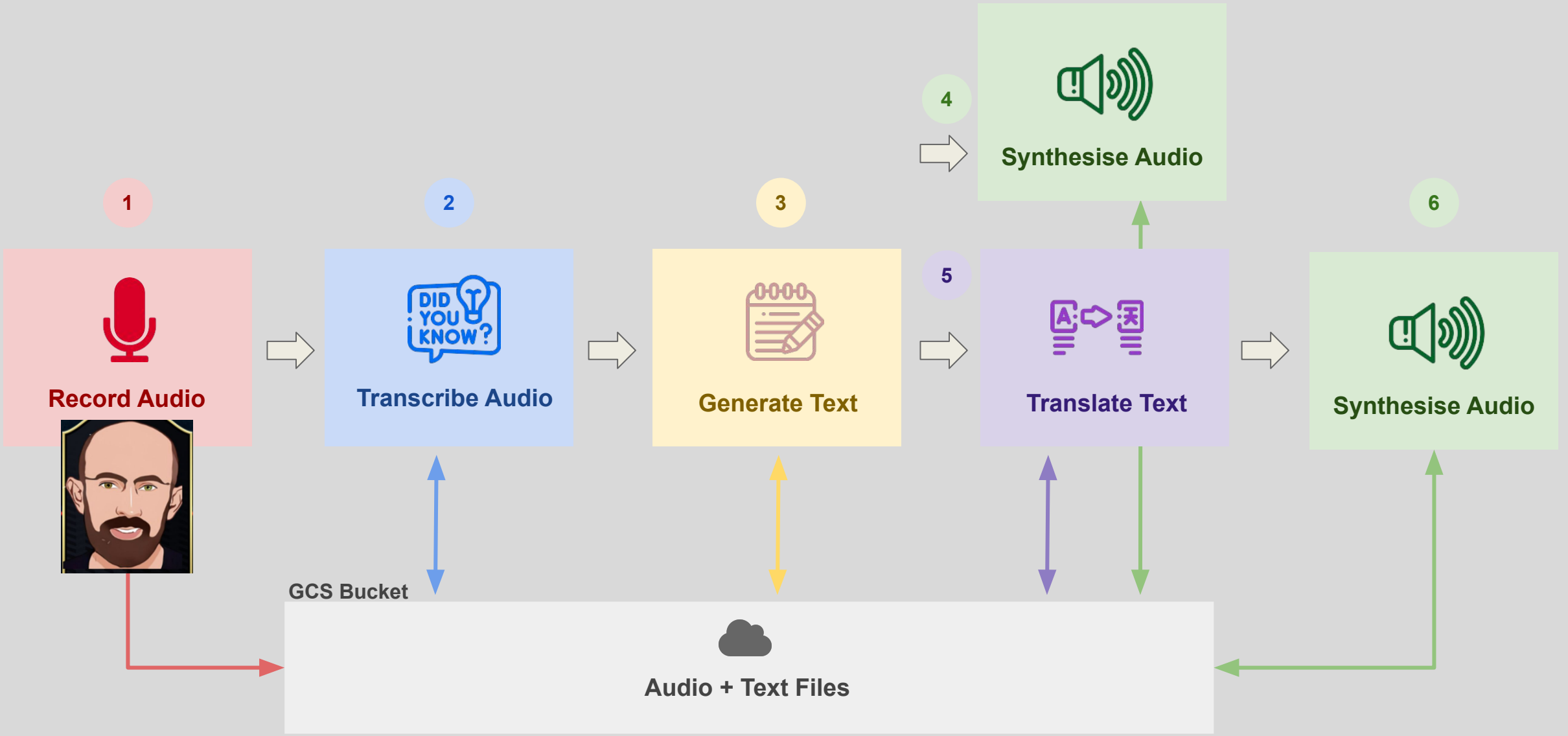
For external identities to access GCP securely.

Tutorial - Building the Mega Pipeline App

- Create a cheese-themed podcast for a global audience.
- Implement a workflow including recording, transcribing, grammar correction/text enhancement via a language model, and audio synthesis.
- Translate podcast content into multiple languages to reach international users.
- Develop the system using a containerized architecture with separate teams for each component.
- Optional: Utilize the ElevenLabs API to generate audio using custom voice models.



Tutorial - Building the Mega Pipeline App



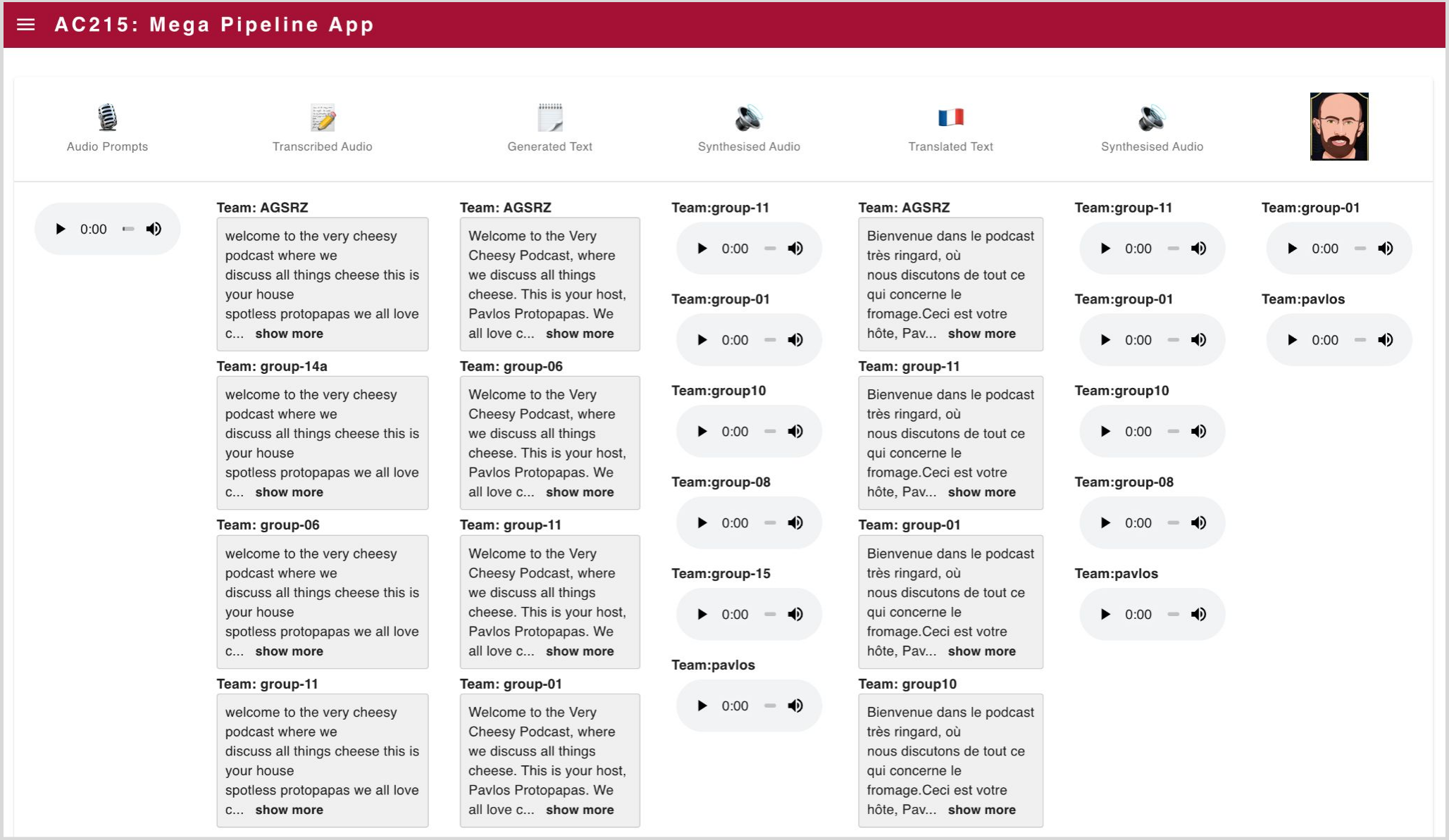
Tutorial - Team Challenge!

- We'll form teams of 5, and each team must complete 5 tasks.
- The first team to finish all tasks will win a special prize!
- Don't worry, every team that completes the tasks will also get a reward!
- The rewards are a surprise—so give it your best effort and have fun!



You get a reward and you get a reward and you....

Tutorial (T5) - Building the Mega Pipeline App



Tutorial - Building the Mega Pipeline App

- App: http://ac215-mega-pipeline.dlops.io/
- Teams
  - 📝Task A transcribe_audio:
  - 🗒️Task B generate_text:
  - 🔊Task C synthesis_audio_en:
  - 🇫🇷Task D translate_text:
  - 🔊Task E synthesis_audio:
- Instructions: https://github.com/dlops-io/mega-pipeline




PRETTY
SHOWS
Delete! Delete!

Outline

1. Recap: Review of Previous Material
2. Containers in Architecture: Microservices vs. Monolithic
3. Tutorial
4. **Build Optimization**
5. Docker Compose
6. Networking
7. Team Workflows

Build Optimization: Why Bother?

Every deploy is a download:

Push to the registry, pull on each server before it can start.

More packages = more to secure & patch:

Build tools and secrets shouldn't ride along.

Slow builds tax the whole team:

Every code change: rebuild → push → pull.

Storage & bandwidth aren't free:

One fat image × every version × every server.

Build Optimization: Why Bother?

Every deploy is a download:

Push to the registry, pull on each server before it can start.

Mor

E

Slo

E

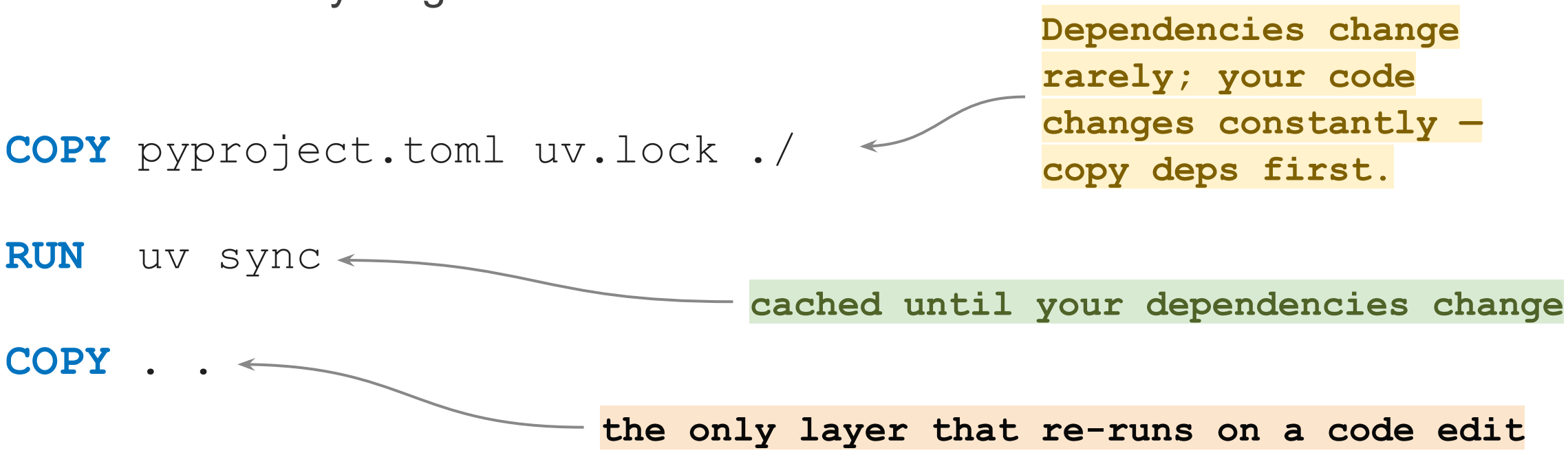
Storage & bandwidth aren't free:

One fat image × every version × every server.

Ship only what you need to RUN, and keep what changes small.

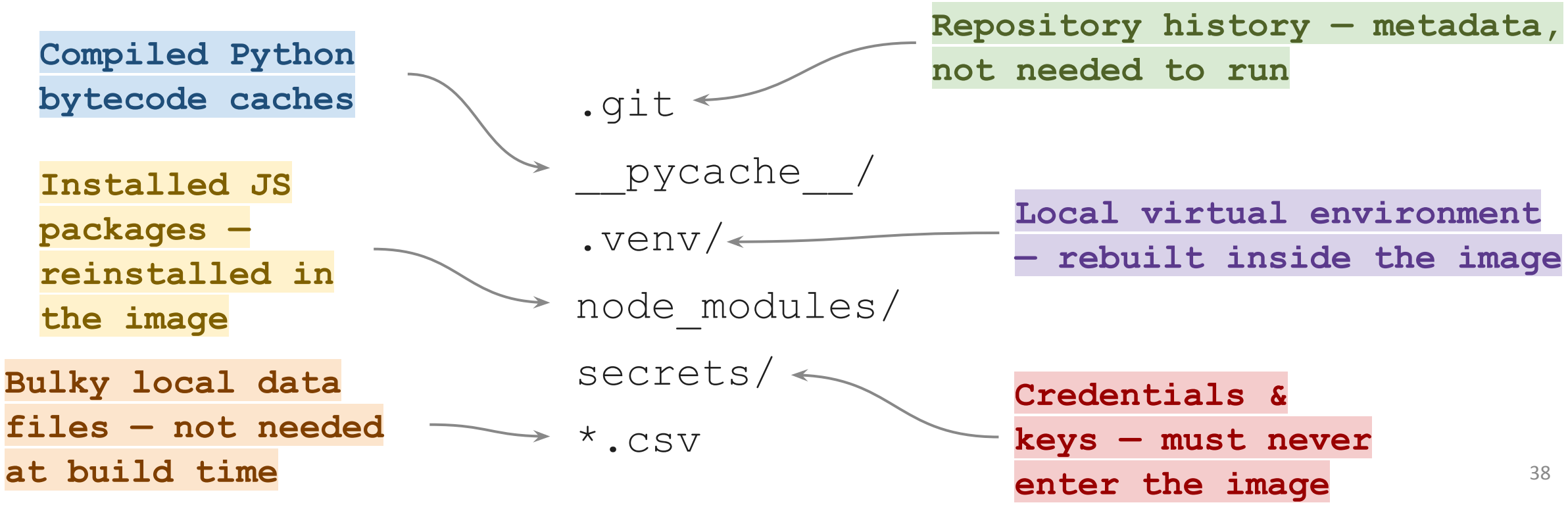
Layer Caching: Order Matters

- Each Dockerfile instruction is a cached layer. Change one layer and you bust the cache for every layer after it.
- So put rarely-changing steps first, frequently-changing steps last.
- Classic mistake: `COPY . .` before installing deps → every code edit reinstalls everything.



`.dockerignore`: Shrink the Build Context

- Like `.gitignore`, but for the **build context** Docker sends to the daemon.
- Each line is a pattern — a file or directory to exclude from the build context.
- Smaller context = faster builds, and you cannot accidentally bake secrets or junk into the image.



How Python Packages Are Installed

- **Prebuilt wheel** (.whl) — already built; pip just unpacks it. No compiler needed. Most packages, including NumPy and pandas.
- **Source** (sdist) — source code, e.g., Python, C, or Rust. If no wheel matches your platform, pip builds one from source. C/Rust extensions require a build toolchain.

Source distribution

`cheese_utils-1.0.tar.gz`

```
pyproject.toml
src/
  cheese_utils/
    __init__.py
    prices.py
tests/
  test_prices.py
```



Built wheel

`cheese_utils-1.0-py3-none-any.whl`

```
cheese_utils/
  __init__.py
  prices.py
cheese_utils-1.0.dist-info/
  METADATA
  WHEEL
  RECORD
```

Build: prepare files for installation

Pick a Smaller Base Image

How a dependency installs — and why the base image matters:

- **Prebuilt wheel** (`.whl`) — already built; pip just unpacks it. No compiler needed. Most packages, incl. numpy & pandas.
- **Source** (sdist) — source code, e.g., Python, C, or Rust. If no wheel matches your platform, pip builds one from source. C/Rust extensions require a build toolchain.

Base image	Approx. size	When to use it
`python:3.14`	**~1.63 GB**	Rarely — its full toolchain can compile from source
`python:3.14-slim`	**~217 MB**	**Default — enough when your deps have wheels**
`python:3.14-alpine`	**~86 MB**	Smallest, but musl → often no wheel, so it compiles from source

Slim for most Python; alpine only when you know what you're doing. Smaller base → fewer known vulnerabilities (CVEs).

Pick a Smaller Base Image

How a dependency installs — and why the base image matters:

- **Prebuilt wheel** (.whl): A ZIP archive containing a ready-to-install package. Pip extracts and installs it. No build tools needed.
- **Source package** (sdist): An archive containing source code. Pip extracts it, builds a wheel, then installs that wheel. Building may require extra build tools.

Base image	Approx. size	When to use it
`python:3.14`	**~1.63 GB**	Build tools included: gcc, g++, make
`python:3.14-slim`	**~217 MB**	**Good default with compatible wheels**
`python:3.14-alpine`	**~86 MB**	Smaller base. Check that your dependencies support Alpine

Multi-stage builds: Use build tools in the build stage. Leave them out of the final image.

Measure It

Optimize what is actually big — do not guess.

`docker images` — the total size of each image.

```
$ docker images python
REPOSITORY  TAG          SIZE
python      3.14         1.63GB
python      3.14-slim    217MB
```

same Python, a leaner base.

`docker history <image>` — per-layer size; find the fat layer.

```
$ docker history python:3.14-slim
CREATED BY                          SIZE
CMD ["python3"]                       0B
RUN … command symlinks             16.4kB
RUN … build Python + pip           45.2MB
ENV PYTHON_VERSION=3.14.7             0B
RUN … apt-get; runtime deps        13.1MB
ENV PATH, PYTHON_SHA256               0B
# debian.sh … (base image)         109MB
```

Multi-Stage Builds

```
FROM python:3.14 AS builder
WORKDIR /app
RUN pip install uv
COPY pyproject.toml uv.lock ./
RUN  uv sync

FROM python:3.14-slim AS runtime
WORKDIR /app
COPY --from=builder /app/.venv /app/.venv
COPY . .
CMD ["/app/.venv/bin/python", "app.py"]
```

Builder stage: Includes common build tools. Uses uv to install dependencies into .venv.

Runtime stage: Starts from slim. Contains only what the app needs to run.

COPY --from=builder: Copies the prepared .venv. Build tools stay in the builder stage.

Build in one stage. Copy what the app needs into the final stage. Build tools stay out of the final image.

Fewer, Leaner Layers

Layers are additive: files deleted in a later layer still take up space in the earlier one.

Trim the extras: --no-install-recommends for apt, --no-cache-dir for pip.

Build-essential: common C/C++ build tools for dependencies built from source.

```
FROM python:3.14-slim
RUN apt-get update \
 && apt-get install -y --no-install-recommends
build-essential \
 && rm -rf /var/lib/apt/lists/*
```

Chain related commands and clean up in the same layer with &&.

A separate cleanup layer does not shrink the image – Delete the downloaded package lists in the same RUN step to save space. The installed build tools remain available.

Building for the Target Platform

Docker build already uses Buildx and BuildKit. Choose the platform where your app will run.

--platform builds for the target's CPU arch, not your laptop's.

```
$ docker build --platform linux/amd64 -t myapp --load .
```

--load puts the image into your local Docker (default).

```
$ docker build --push \
      --platform linux/amd64,linux/arm64 -t myapp .
```

--push: Push the multi-platform image to a registry.

Apple Silicon = arm64
Intel / AMD = amd64
Clouds = arm64 or amd64
Check the deployment target.

Build Secrets: Tokens for Dockerfile RUN Steps

When a Dockerfile RUN step needs a token, supply it through a temporary secret mount so the token itself stays out of the Dockerfile and image.

```
# syntax=docker/dockerfile:1
RUN --mount=type=secret,id=gh_token \
    GH_TOKEN=$(cat /run/secrets/gh_token) &&\

    pip install "git+https://GH_USERNAME:${GH_TOKEN}@github.com/org/lib"
```

Enables --mount — must be the Dockerfile's first line.

Names the secret — reuse this id on docker build.

A container path — mounted here only for this build step.

$(…) runs cat so it reads that file into GH_TOKEN.

Build Secrets: Supply the Token from Your Terminal

Supply the token file when starting the build, using the same secret ID as the Dockerfile RUN step.

A local path — the token file on your machine.

Same id as the RUN --mount — that's the link.

```
$ docker build --secret id=gh_token,src=$HOME/.gh_token .
```

Avoid these ways of passing secrets:

Anti-pattern	Risk for secrets
`ENV GH_TOKEN=ghp_xxx`	Stored in image configuration
`ARG GH_TOKEN`	May appear in history or build metadata
`COPY .gh_token /tmp/token`	Remains in earlier layer after later deletion

Outline

1. Recap: Review of Previous Material
2. Containers in Architecture: Microservices vs. Monolithic
3. Tutorial
4. Build Optimization
5. **Docker Compose**
6. Networking
7. Team Workflows

The Problem Compose Solves

A real app is many containers — starting them by hand is a chore:

- One `docker run` per service — each with its own `-p`, `-v`, `-e`, `--network` flags.
- You create the network and start the services in the right order, yourself.
- Every code change: stop → remove → rebuild → run, service by service.

Our example — a two-container UPPER-case transformer:




```
docker network create app-net
docker build -t processor-img ./processor
docker run -d --network app-net --name processor processor-img
docker build -t gateway-img ./gateway
docker run -d --network app-net -p 8090:9100 \
  -e PROCESSOR_HOST=processor -e PROCESSOR_PORT=9200 \
  -v "$PWD/gateway:/app" gateway-img
```

The Problem Compose Solves

A real app is many containers — starting them by hand is a chore:

- One `docker run` per service — each with its own `-p`, `-v`, `-e`, `--network` flags.
- You create the network and start the services in the right order, yourself.
- Every code change: stop → remove → rebuild → run, service by service.

One multi-container app deserves one file and one command.

What Is Docker Compose?

One tool + one file to define and run a multi-container app.

- **One file** — `compose.yaml` describes every service, network, and volume (legacy name: `docker-compose.yaml`).
- **One tool** — `docker compose`, built into Docker.
- You state the end result; Compose builds, creates, and starts it.

Task	By hand	With Compose
Network	`docker network create + attach each`	**automatic, shared**
Start it all	`N x docker run (in order)`	**`docker compose up`**
Tear down	`stop + rm each`	**`docker compose down`**

compose.yaml

Compose is `docker run`, lifted out of your shell into a file you can read, diff, and re-run.

```yaml
services:
  gateway:
    build: ./gateway
    ports:
      - "8090:9100"
    volumes:
      - ./gateway:/app
    environment:
      PROCESSOR_HOST: processor
      PROCESSOR_PORT: "9200"
    depends_on:
      - processor

  processor:
    image: ac215/processor:1.0
```

One service per entry — name is its hostname

ports (-p)

volumes (-v)

environment (-e)

build: vs image:

`build` vs `image` — each service picks where its image comes from.

```yaml
services:
  gateway:
    build: ./gateway
    ports:
      - "8090:9100"
    volumes:
      - ./gateway:/app
    environment:
      PROCESSOR_HOST: processor
      PROCESSOR_PORT: "9200"
    depends_on:
      - processor

  processor:
    image: ac215/processor:1.0
```

Compose builds an image from ./gateway using its Dockerfile and local files.

Pull a prebuilt image from a registry, such as Docker Hub.

Networking for Free

Your gateway calls the processor by its name — you just ran it:



You never ran this:

```
$ docker network create app-net
$ docker run --network app-net --name gateway …
```

Compose creates the network for you.

Volumes

Attach storage to a service — just a list under the `volumes:` key.

```yaml
services:
  gateway:
    build: ./gateway
    ports:
      - "8090:9100"
    volumes:
      - ./secrets:/secrets:ro
      - models:/models
    environment:
      PROCESSOR_HOST: processor
      PROCESSOR_PORT: "9200"
    depends_on:
      - processor

volumes:
  models:
```

Bind mount — host path in

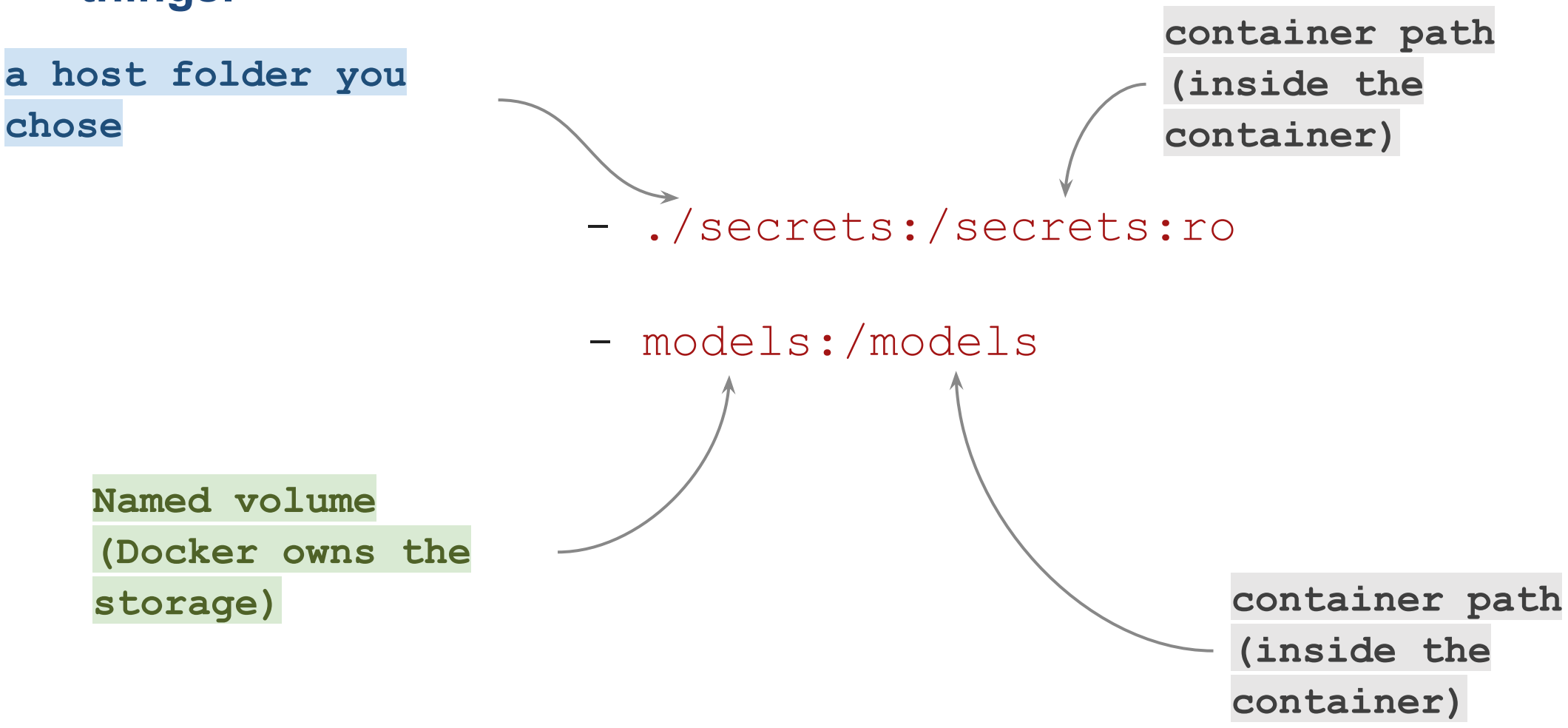
:ro = read-only

Named volume — Docker persists it, just a name

Declare the named volume here

Volumes: Bind Mount vs Named Volume

Same `source:container` shape — but the left side means two different things.



Environment

Two ways to pass variables into a service — both injected at run time.

```yaml
services:
  gateway:
    build: ./gateway
    ports:
      - "8090:9100"
    volumes:
      - ./gateway:/app
    environment:
      PROCESSOR_HOST: processor
      PROCESSOR_PORT: "9200"
    env_file:
      - .env
```

Set vars inline, like -e (a few non-secret values)

Load a file (.env) kept out of git; secrets stay out of the YAML

Environment: Build Time vs Run Time

Where you write a variable decides when it exists.

Where you write it	Compose file (${VAR})	Build	Image	Container (run)
`Dockerfile: ENV`	—	✓	✓	✓
`Dockerfile: ARG`	—	✓	—	—
`Compose: build.args`	—	✓	—	—
`Compose: environment: / env_file:`	—	—	—	✓
`.env file (auto-loaded)`	✓	—	—	—
`Shell variable $VAR`	✓	—	—	—

```
# .env
TAG=1.0

# Or in the shell
export TAG=1.0
```

fills →

```
#docker-compose.yml
services:
  processor:
    image: ac215/processor:${TAG}
```

Startup Order

You've seen `gateway` depend on `processor` — what does it guarantee?

```yaml
services:
  gateway:
    build: ./gateway
    ports:
      - "8090:9100"
    volumes:
      - ./gateway:/app
    environment:
      PROCESSOR_HOST: processor
      PROCESSOR_PORT: "9200"
    depends_on:
      - processor

  processor:
    image: ac215/processor:1.0
```

Processor comes "up" first, then gateway.

But "up" = container started, not app ready.

Startup Order

Real readiness: wait until processor is healthy, not just started.

```yaml
services:
  gateway:
    build: ./gateway
    ports:
      - "8090:9100"
    volumes:
      - ./gateway:/app
    environment:
      PROCESSOR_HOST: processor
      PROCESSOR_PORT: "9200"
    depends_on:
      processor:
        condition: service_healthy

  processor:
    image: ac215/processor:1.0
    healthcheck:
      test: curl -f localhost:9200/health
```

A fixed Compose value, not a name you invent.
The other two:
service_started
service_completed_successfully

Order — processor comes "up" first, then gateway.

hold gateway until the check passes.

healthcheck — the probe that decides when processor is ready.

Everyday Compose Commands

Run these from the directory holding your `docker-compose.yml`:

```
docker compose up
docker compose up -d
docker compose up --build
docker compose ps -a
docker compose logs -f
docker compose down
docker compose down -v
```

`up` — build images, create the network, and start every service.

`-d` — run detached, in the background.

`--build` — rebuild the image first.

`ps -a` — which services are up, and which have stopped.

`logs -f` — watch every service's output live, in one stream.

`down` — stop and remove all containers and the network at once.

`-v` — also delete named volumes (your data). Delete!

The Whole App in One File

```yaml
services:
  gateway:
    build: ./gateway
    ports:
      - "8090:9100"
    volumes:
      - ./gateway:/app
    environment:
      PROCESSOR_HOST: processor
      PROCESSOR_PORT: "9200"
    depends_on:
      - processor

  processor:
    image: ac215/processor:1.0
```

The whole app — `gateway` + `processor` — in one file. `docker compose up` runs all of it.

When Compose Isn't Enough

- Compose runs on **one machine** — great for dev and a single small server.
- No multi-machine scaling, self-healing, rolling updates, or cross-node load balancing.
- Production, multi-host → **Kubernetes** (later in the course) — same “declare services, find by name” idea.

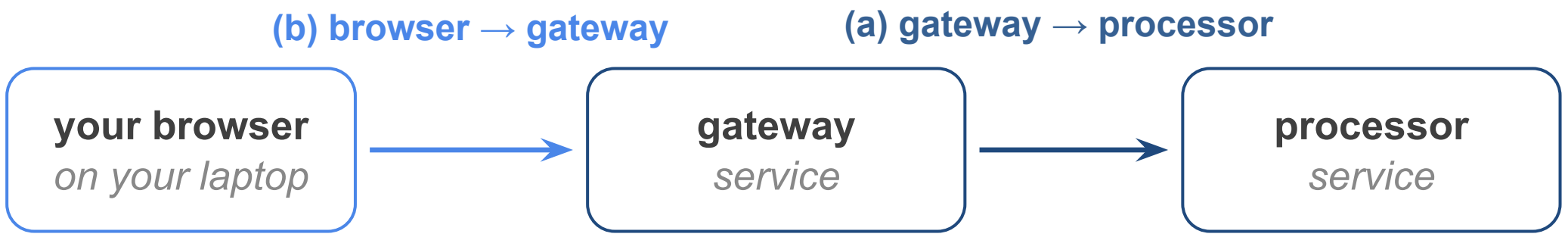
Aspect	Docker Compose	Kubernetes
Scope	one machine	many machines (a cluster)
Great for	dev + a single small server	production scale, self-healing, rolling updates

Outline

1. Recap: Review of Previous Material
2. Containers in Architecture: Microservices vs. Monolithic
3. Tutorial
4. Build Optimization
5. Docker Compose
6. **Networking**
7. Team Workflows

Your Containers Need to Talk

Compose starts many containers — but two questions decide whether anything works:

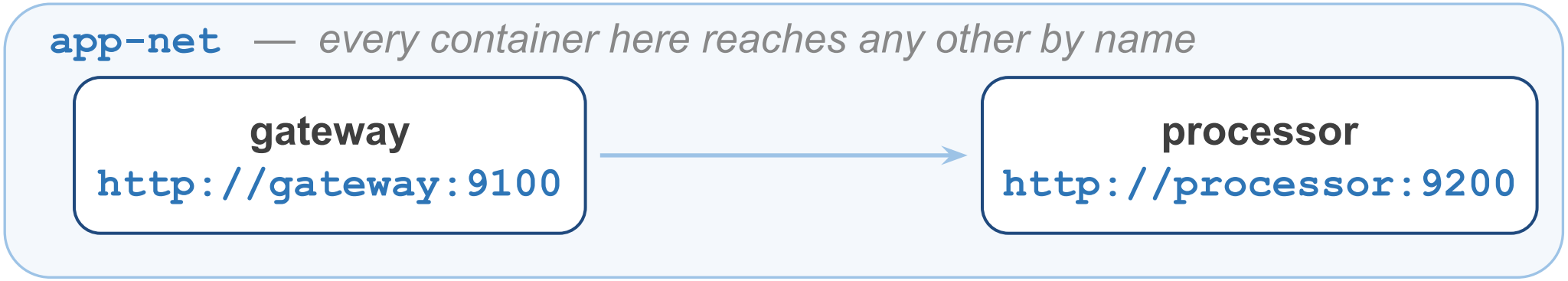


Arrows show request direction; responses return over the same connections.

- **(a) Gateway → processor** — one service needs to reach another.
- **(b) Browser → gateway** — you, at your browser, need to reach the app from outside.
- **Isolated by default** — each container has its own network namespace (Containers I) until you wire these paths.

Gateway → Processor: One Shared Network

Put them on one network → they reach each other by name.




```
1 — create one shared network
$ docker network create app-net

2 — attach each container & name it
$ docker run --network app-net \
      --name gateway    gateway-img
$ docker run --network app-net \
      --name processor  processor-img

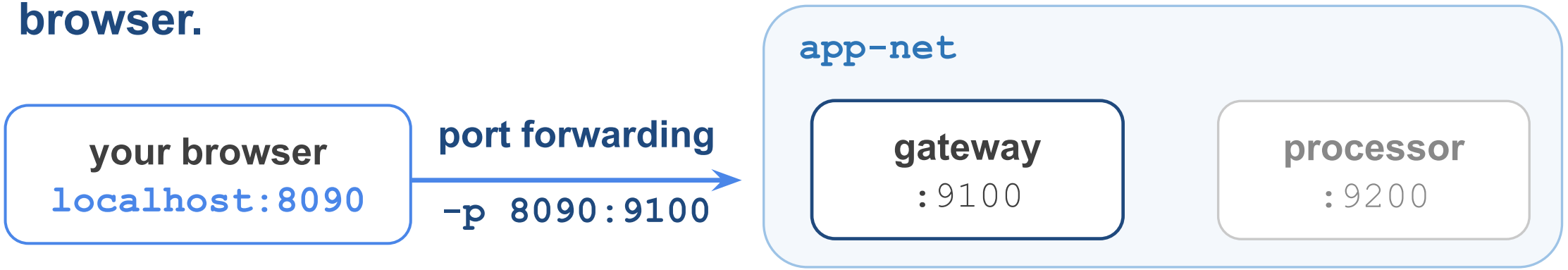
3 — reach it by name — Docker DNS, no IP
POST http://processor:9200/
```

Optional here: no container calls gateway by name.

Compose does all 3 for free — It creates the network, connects the containers, and makes services reachable by name.

Browser → Gateway: Publish a Port

Publish a port to reach a container from your laptop — open its page in a browser.



8090 : 9100 = HOST : CONTAINER

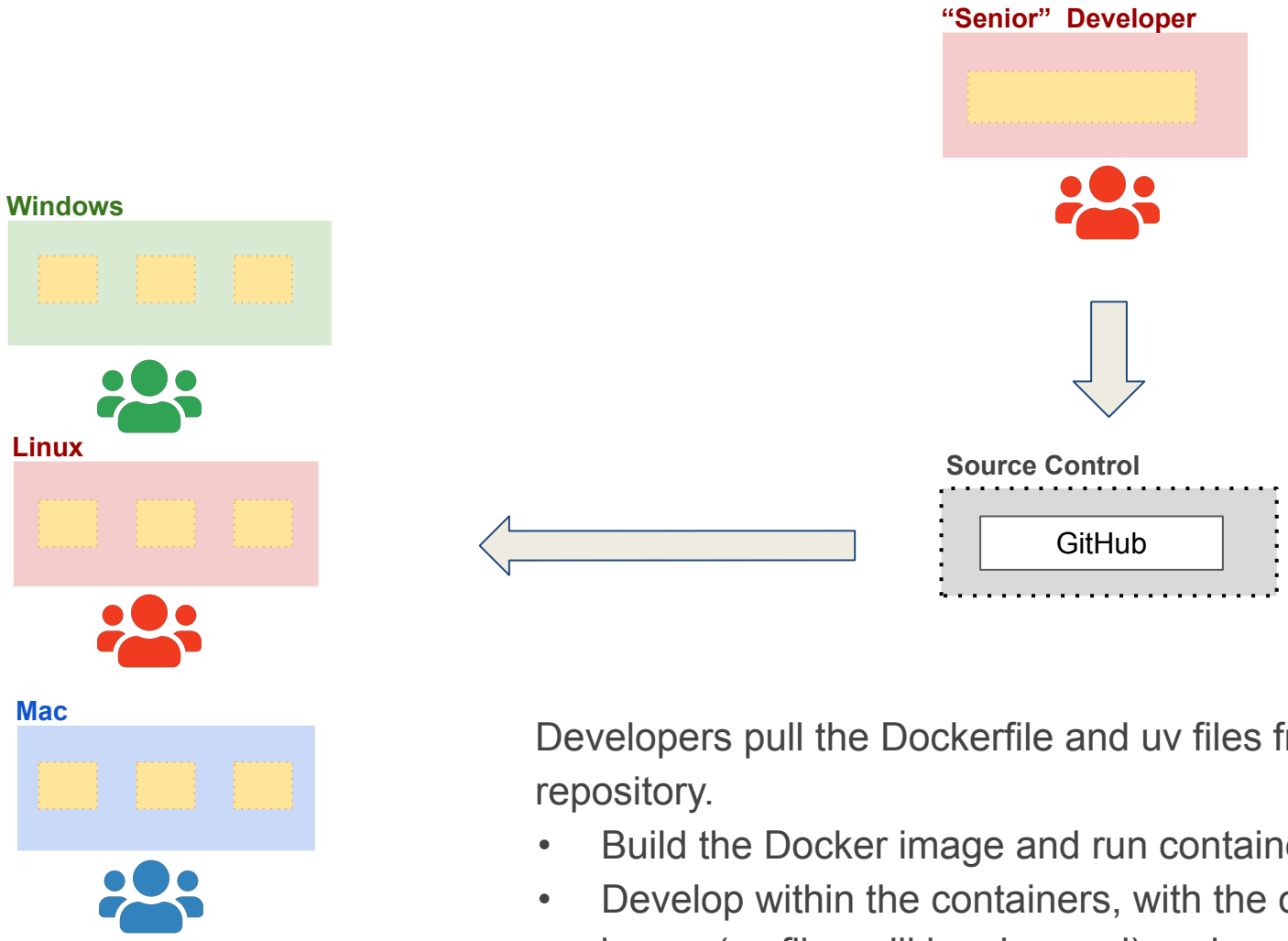
```
# docker run command
$ docker run --network app-net \
        --name gateway \
    -p 127.0.0.1:8090:9100 gateway-img
```

```
# Compose ports excerpt
    ports:
      - "127.0.0.1:8090:9100"
```

Outline

1. Recap: Review of Previous Material
2. Containers in Architecture: Microservices vs. Monolithic
3. Tutorial
4. Build Optimization
5. Docker Compose
6. Networking
7. **Team Workflows**

Workflow with Docker: Scenario 1 (early stages of development)



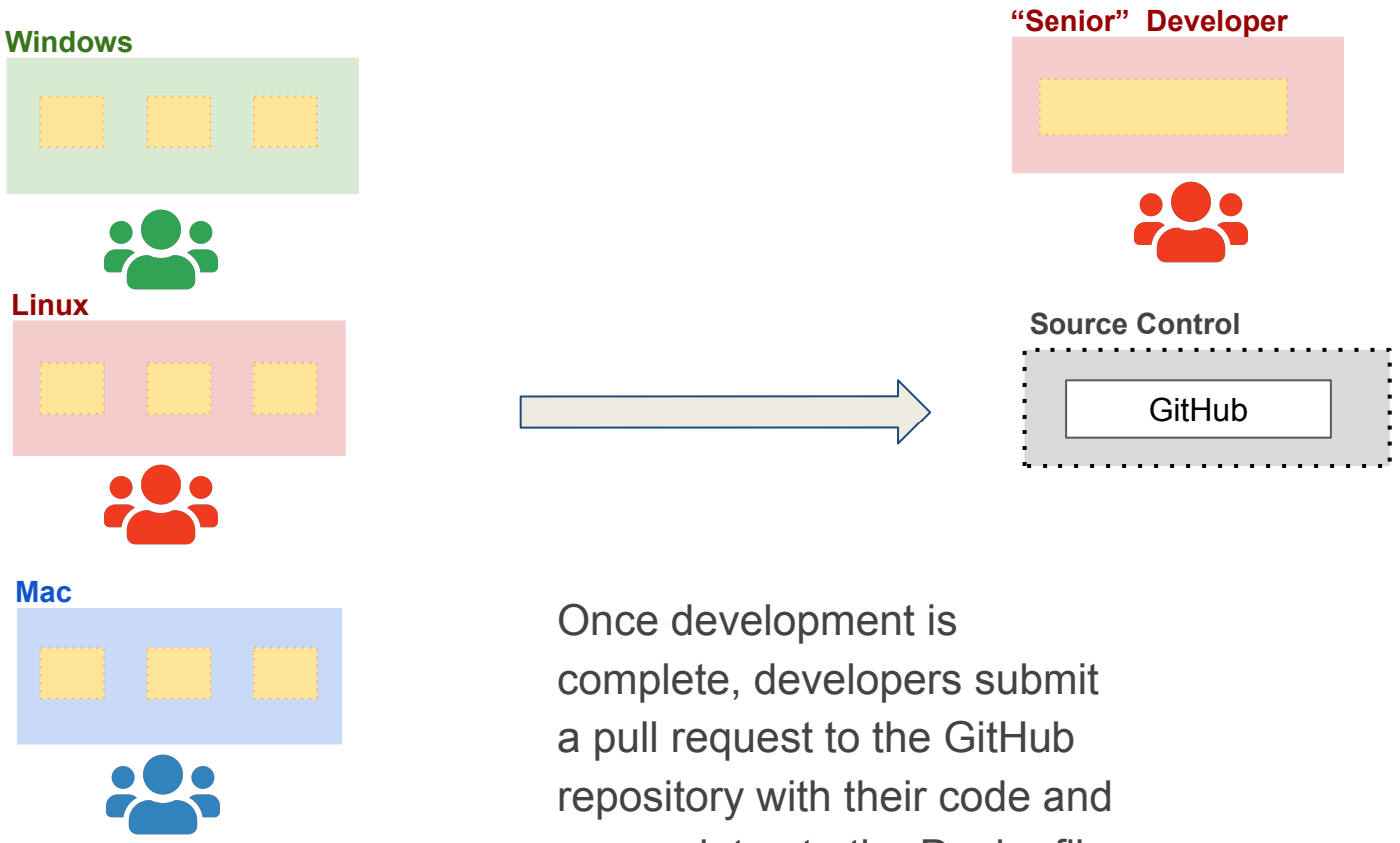
A senior developer is responsible for creating the Dockerfile, along with the initial pyproject.toml and uv.lock.

Push Dockerfile and uv files to the GitHub repository

Developers pull the Dockerfile and uv files from the GitHub repository.

- Build the Docker image and run containers.
- Develop within the containers, with the option to install packages (uv files will be changed) and modify the Dockerfile if needed, then rebuild.

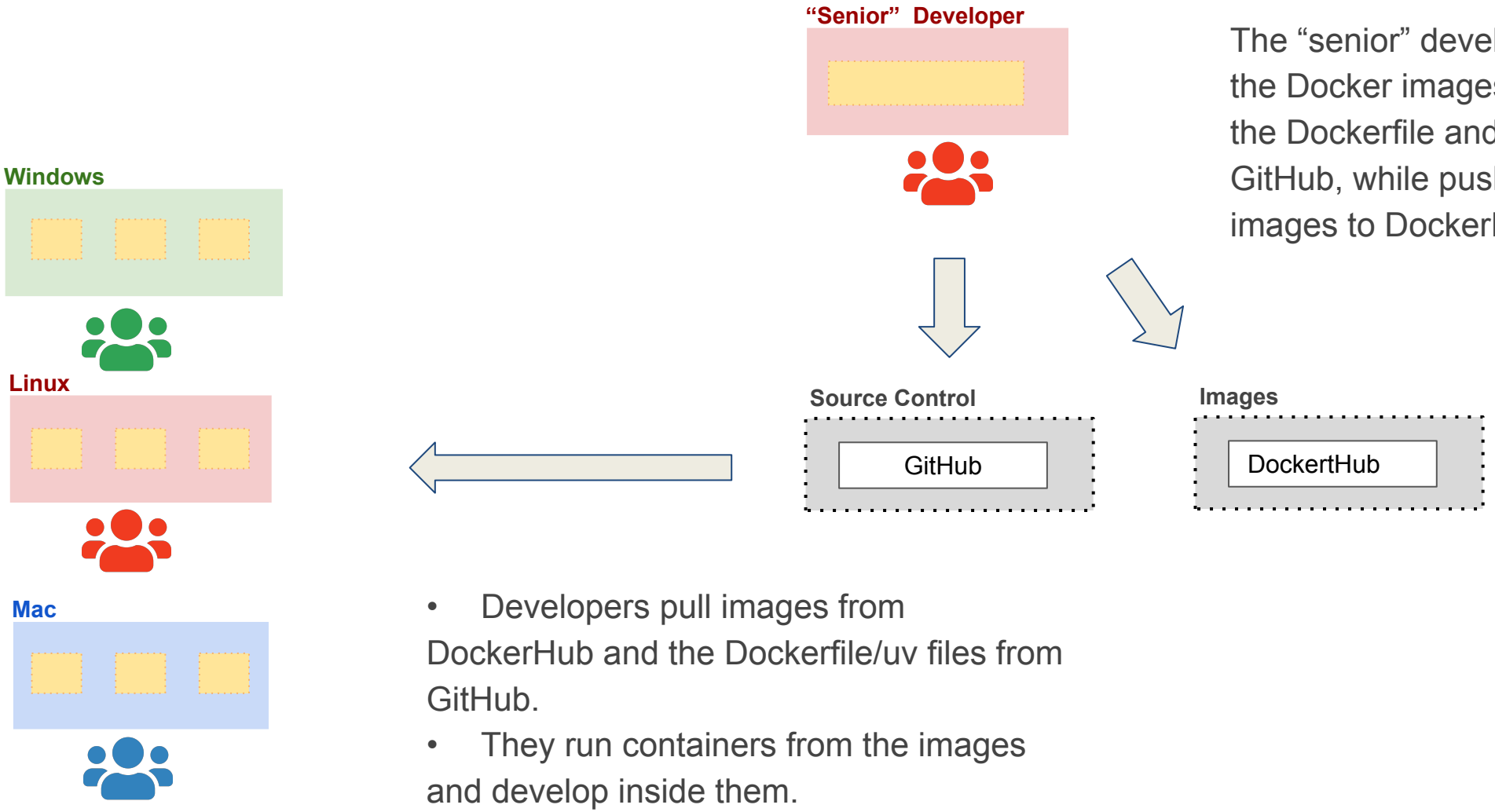
Workflow with Docker: Scenario 1 (early stages of development)



Once development is complete, developers submit a pull request to the GitHub repository with their code and any updates to the Dockerfile or uv files.

The senior developer reviews and merges all pull requests, then consolidates the updates into a new Dockerfile and uv files.

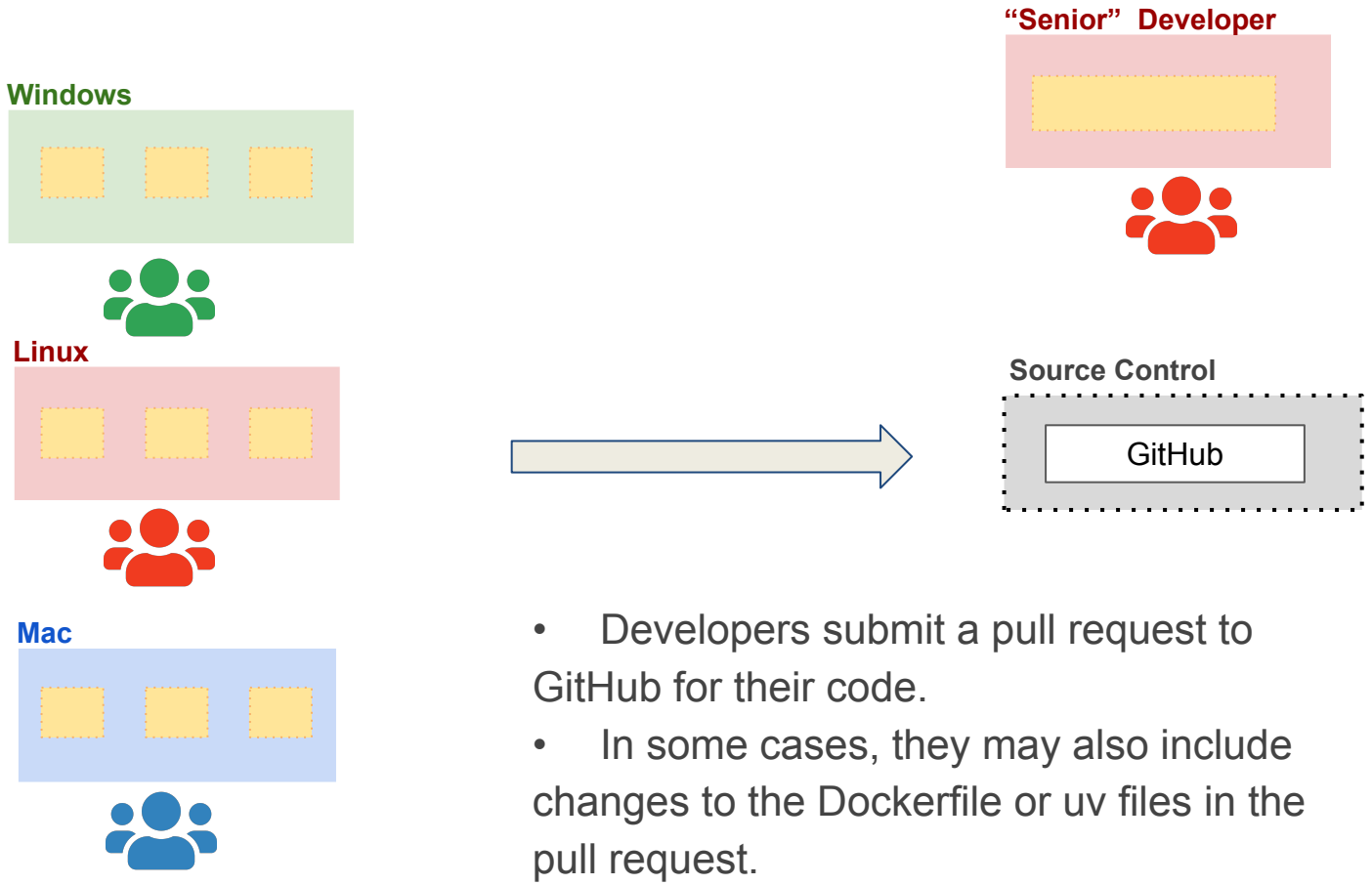
Workflow with Docker: Scenario 2 (later stages of development)



The "senior" developer creates the Docker images and pushes the Dockerfile and uv files to GitHub, while pushing the images to DockerHub.

- Developers pull images from DockerHub and the Dockerfile/uv files from GitHub.
- They run containers from the images and develop inside them.
- Rebuilding the images is only needed on rare occasions.

Workflow with Docker: Scenario 2 (later stages of development)



The senior developer reviews and merges all pull requests, then consolidates the updates into a new Dockerfile and uv files and builds new images which are pushed to DockerHub.

- Developers submit a pull request to GitHub for their code.
- In some cases, they may also include changes to the Dockerfile or uv files in the pull request.

Workflow with Docker: A Flexible Approach

Is there a “perfect” workflow?

No

So, how do we decide what to do?

Clear communication and rules are essential. Each team can have its own workflow, based on the project and team needs.

Next time: you'll need real authentication

- In Tutorial 5 we just **handed out** a shared service-account key and secrets — fine for a class demo, not for the real world.
- To pull images from a **private registry**, read and write **cloud storage**, or call a Google API, your containers must prove **who they are**.
- Next lecture — **Connecting to the Cloud** — is real auth: API keys, OAuth 2.0, **service accounts** & IAM.
- Plus the networking behind `-p` and `localhost`, and cloud storage (GCS & Drive).
- You'll trade "secrets handed out" for **identities you create and control**.



THANK YOU